

VMS Audit — Executive Summary

[object Object],[object Object]
[object Object],[object Object]
[object Object],[object Object]

Current Architecture

- **Frontend:** React 19 + Vite 8, Tailwind CSS v4, react-router-dom v7, plain JavaScript (no TypeScript). Icons via lucide-react, dates via date-fns.
- **Backend / Data:** Firebase (Firestore + Auth, Web SDK v12). All logic runs client-side in the browser.
- **Email:** EmailJS, called directly from the browser (keys exposed to clients).
- **Face detection:** face-api.js (tiny-face-detector) runs in-browser for auto photo capture.
- **Backend (unused):** `functions/` contains Firebase Cloud Functions (nodemailer expiry emails) with placeholder credentials. **Not deployed** — the project is hosted on Netlify, which cannot run them.
- **Deployment:** Netlify (`netlify.toml`) with SPA redirects to `index.html`.
- **Configuration:** Firebase + EmailJS credentials are read from `.env` (`VITE_FIREBASE_*`, `VITE_EMAILJS_*`) and are currently set.

[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],
[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],
[object Object],[object Object],[object Object],[object Object]

Current Workflow (Visitor Self-Registration)

[object Object],[object Object],[object Object],[object Object],[object Object]
[object Object]
[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]
[object Object]
[object Object]
[object Object],[object Object],[object Object],[object Object],[object Object]
[object Object]

Reusable Components (REUSE)

Asset	Value to the redesign
AuthContext, AdminRoute, AdminLogin, ResetPassword	Complete Firebase Auth stack (sign-in, sign-up, role load, password recovery) — extend with a Host role
lib/roles.js + ROUTE_ACCESS	Role model and route guard matrix — extend with a Host role
Sidebar, AdminLayout	Admin portal shell (navigation, header, responsive)

AdminDashboard, AdminVisitors, AdminReports	Analytics, history table, and CSV export (rebuild queries against the new status model)
ExtendVisitModal, StayMonitor	Visit extension and expiry-reminder logic
lib/visitUtils.js, lib/settings.js	Duration/status helpers and app settings
lib/email.js	EmailJS wrapper (keep as client fallback or port server-side)
Camera + face-capture logic (in CheckInForm)	Extract into a reusable photo-capture component for reception check-in
firebase.rules structure	Role helper functions (isSignedIn, getRole, isStaff) to extend
CSS token/theme system, Tailwind v4	Keep as the design foundation

Components Requiring Modification (MODIFY)

- `**AdminHosts**` — becomes the core registration driver: book visitors against a host and show a host's visitors.
- `**AdminPreRegistrations**` — align with the host-driven booking lifecycle (badge + QR generated at booking time).
- `**DigitalBadge**` — add QR code, pre-arrival "expected" badge state, and registration number.
- `**App.jsx**` — add `/host/*` and reception routes.
- `**StayMonitor**` — adapt to the new status enum (``Pre-Registered`` `!` ``Checked In`` `!` ``Checked Out``).
- `**AdminNotifications**` — notification history for registration, arrival, check-in, and check-out.
- `**firebase.rules**` — add Host role, host write rules, reception check-in rules.
- `**roles.js / `AuthContext**` — add the Host role and host account creation.

Components Requiring Rebuild (REBUILD)

- **Host registration workflow** — host logs in, registers a visitor (purpose, duration, expected arrival), submits, and a badge + QR is generated with confirmation emails.
- **Reception check-in flow** — search pre-registered visitor `!` verify ID (record verifier) `!` capture photo `!` check in `!` notify host. No equivalent exists today.
- **Badge generation with QR** — no QR library is present; needs new generation logic and a stored, scannable payload.
- **Host portal / dashboard** — hosts log in to see their registrations, arrival status, and check-in confirmations.
- **Check-out flow** — today only a manual admin button; needs reception/host/auto options with timestamps.
- **Photo storage** — base64-in-Firestore is near the 1 MiB limit; move to Firebase Storage with download URLs.

[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]

Database Changes Required

- **Unify the visit lifecycle** in one collection (prefer `visitors`): status enum `Pre-Registered` `Checked In` `Checked Out`.
- **visitors** additions: `registeredBy` (host uid), `hostId`, `hostName`, `hostEmail`, `registrationTime`, `expectedArrivalTime`, `badgeCode`/`qrPayload`, `idVerified`, `idVerifiedBy`, `idVerifiedAt`, `checkInBy`, `checkOutBy`, `checkOutTime`, `photoPath` (Storage), and per-stage email flags (`registrationEmailSent`, `hostArrivalEmailSent`, `checkInEmailSent`, `checkOutEmailSent`).
- **preRegistrations**: merge into the `visitors` lifecycle or add `hostId`, `registeredBy`, `badgeCode`, `qrPayload`.
- **users**: allow `role`: 'host'; link `hostId` to a `hosts` record.
- **hosts**: add `userId`/`hostUid` to bind a directory record to a login.
- **notifications** (new): log every email sent (to, type, visitorId, timestamp, status).
- **Firebase Storage**: new bucket for visitor photos (SDK not yet initialized).
- **Composite indexes**: add `status + expectedCheckoutTime`, `registeredBy + registrationTime`, `hostId + checkInTime` (index file is currently empty).

Security Changes Required

- **Add the Host role** to `roles.js`, `ROUTE_ACCESS`, rule helpers, and labels.
- **Firestore rules**:
[object Object],[object Object],[object Object]
[object Object],[object Object],[object Object]
[object Object],[object Object],[object Object]
[object Object],[object Object],[object Object],[object Object],[object Object]
[object Object],[object Object],[object Object]
- **Move email server-side** — EmailJS keys are exposed in the browser; registration/arrival/check-in emails should be triggered by a serverless function (Netlify Function or Firebase Function), with a notification log.
- **Storage rules** for visitor photos (authenticated read, host/staff write).

Recommended Implementation Order

[object Object],[object Object],[object Object]
[object Object],[object Object],[object Object],[object Object],[object Object]
[object Object],[object Object],[object Object]
[object Object],[object Object],[object Object],[object Object],[object Object]
[object Object],[object Object],[object Object]
[object Object],[object Object],[object Object]
[object Object],[object Object],[object Object]
[object Object],[object Object],[object Object]